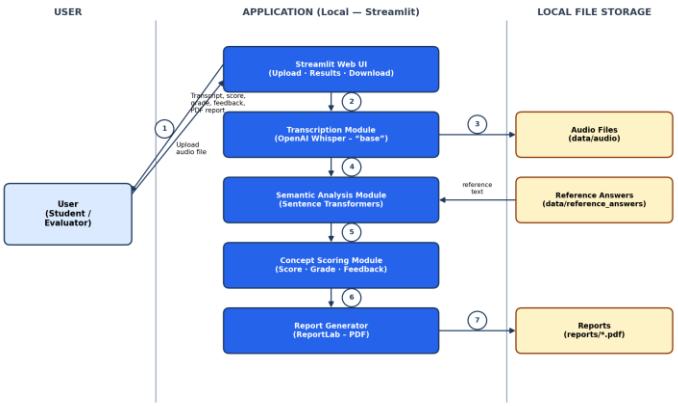
Model Selection & Architecture Design

Date	13 July 2026
Team ID	
Project Name	Voice-Based Concept Understanding Analyzer (VBCUA)

Model Selection & Architecture Design

This section describes the models selected for the Voice-Based Concept Understanding Analyzer (VBCUA), the reasoning behind their selection, the system architecture, and the associated integration, context-handling, tooling, and scalability considerations.

Model Selection & Architecture Design	**Selected model(s):** VBCUA does not use a generative or chat-based large language model such as Gemini Pro/Flash or Vertex AI. It uses two pre-trained, task-specific models: OpenAI Whisper ("base" size) for automatic speech-to-text transcription, and Sentence Transformers (all-MiniLM-L6-v2) for generating text embeddings used in semantic similarity comparison. **Reason for model selection:** Whisper "base" was selected as a lightweight, open-source speech recognition model that runs locally without an external API dependency or associated usage cost, making it suitable for an academic prototype with modest hardware requirements. all-MiniLM-L6-v2 was selected because it is a compact, fast Sentence Transformer model well-suited to short-to-medium length text (such as a spoken concept explanation), offering a good balance between semantic accuracy and inference speed on CPU-only machines. **System architecture diagram:**  *Figure 1: VBCUA System Architecture (Local, Single-Process Deployment)* **Workflow design:** VBCUA does not follow a prompt-based generative workflow (User → Prompt → Model → Output), since neither Whisper nor Sentence Transformers is prompted with instructions. The actual workflow is: User → Audio Upload → Transcription (Whisper) → Semantic Comparison against the reference answer (Sentence Transformers) → Scoring & Grading → Output (on-screen results and downloadable PDF report). **API integration architecture:** Both models run locally within the same Python process as the Streamlit application; there is no integration with an external, cloud-hosted model API (such as Gemini or

	Vertex AI) in the current version. Model classes are loaded directly from the openai-whisper and sentence-transformers Python packages and cached in memory for reuse across a session. **Token / context handling:** Whisper processes raw audio directly and is not subject to a text token limit at the input stage. The Sentence Transformer model (all-MiniLM-L6-v2) has a maximum input sequence length (typically 256 word-pieces); text beyond this length is truncated internally by the model during encoding. Future enhancement: the current implementation does not explicitly chunk or summarise very long transcripts before similarity scoring. For substantially longer spoken responses, adding explicit chunking or truncation-aware handling is identified as a necessary improvement to avoid silently dropping content beyond the model's context window. **Libraries & tools used:** openai-whisper (speech-to-text), sentence-transformers and torch (semantic embeddings and similarity), streamlit (web interface), reportlab (PDF report generation), and python-dotenv, numpy, and related supporting packages listed in requirements.txt. **Scalability considerations:** The current architecture loads both models once per process and caches them in memory (module-level singleton pattern), which avoids reloading overhead within a session but is designed for single-user, single-session use rather than concurrent multi-user access. Future enhancement: scaling to multiple concurrent users would require moving model inference behind a dedicated backend service (rather than within the Streamlit process directly), introducing a request queue, and deploying to a cloud environment with sufficient compute — none of which is implemented in the current version.